

KNOWLEDGE SHARING SESSION

HOW TO BUILD BETTER PRODUCTS

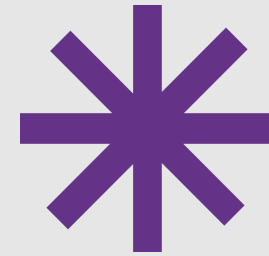


PRESENTED BY : MASHIHA SHRESTHA
NOZOMI GIRI

JULY 13, 2026



ABOUT US



MASHIHA SHRESTHA



NOZOMI GIRI



ABOUT LSPP

Leapfrog Student Partnership Program is a future-oriented program aiming to help IT and engineering students build their career paths and create an impact in the tech community. As a Leapfrog Student Partner, you will serve as an opinion leader and an influencer to engage and connect with.

BENEFITS OF BEING AN LSP

1. All-round learning and personal growth
2. Take a step into the professional world
3. Become a leader and empower your peers
4. Be eligible for exquisite rewards

AS AN LSP

1. Build tech communities by bringing your peers together
2. Host regular workshops and events focused on accelerating skill building
3. Strengthen your own skill set while building future leaders

TABLE OF CONTENT



01 THE REALITY CHECK

02 THINK LIKE A PRODUCT
BUILDER

03 UNDERSTANDING
REQUIREMENTS

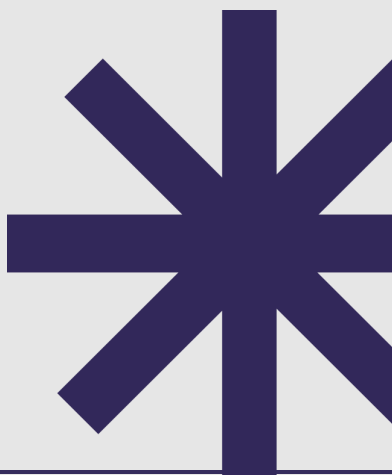
04 PRIORITIZING
REQUIREMENTS

05 USER STORIES AND
PERSONAS

06 ESTIMATION & PLANNING

07 AGILE & HOW TEAMS
WORK

08 GITHUB WORKFLOW



OBJECTIVES

By the end of this session, you will be able to:

- Understand how successful hackathon teams approach problem-solving using product thinking.
- Identify real user problems and distinguish them from assumptions.
- Define user needs through personas, problem statements, and user stories.
- Prioritize features and define a focused solution for a hackathon.
- Collaborate effectively by planning tasks, assigning responsibilities, and estimating work using agile techniques.
- Apply practical tools such as user stories, MoSCoW prioritization, Value vs. Effort Matrix, and Planning Poker.



THE REALITY CHECK

THE REALITY CHECK:

WHAT REALLY HAPPENS IN A HACKATHON



The Starting Phase



The Ending Phase





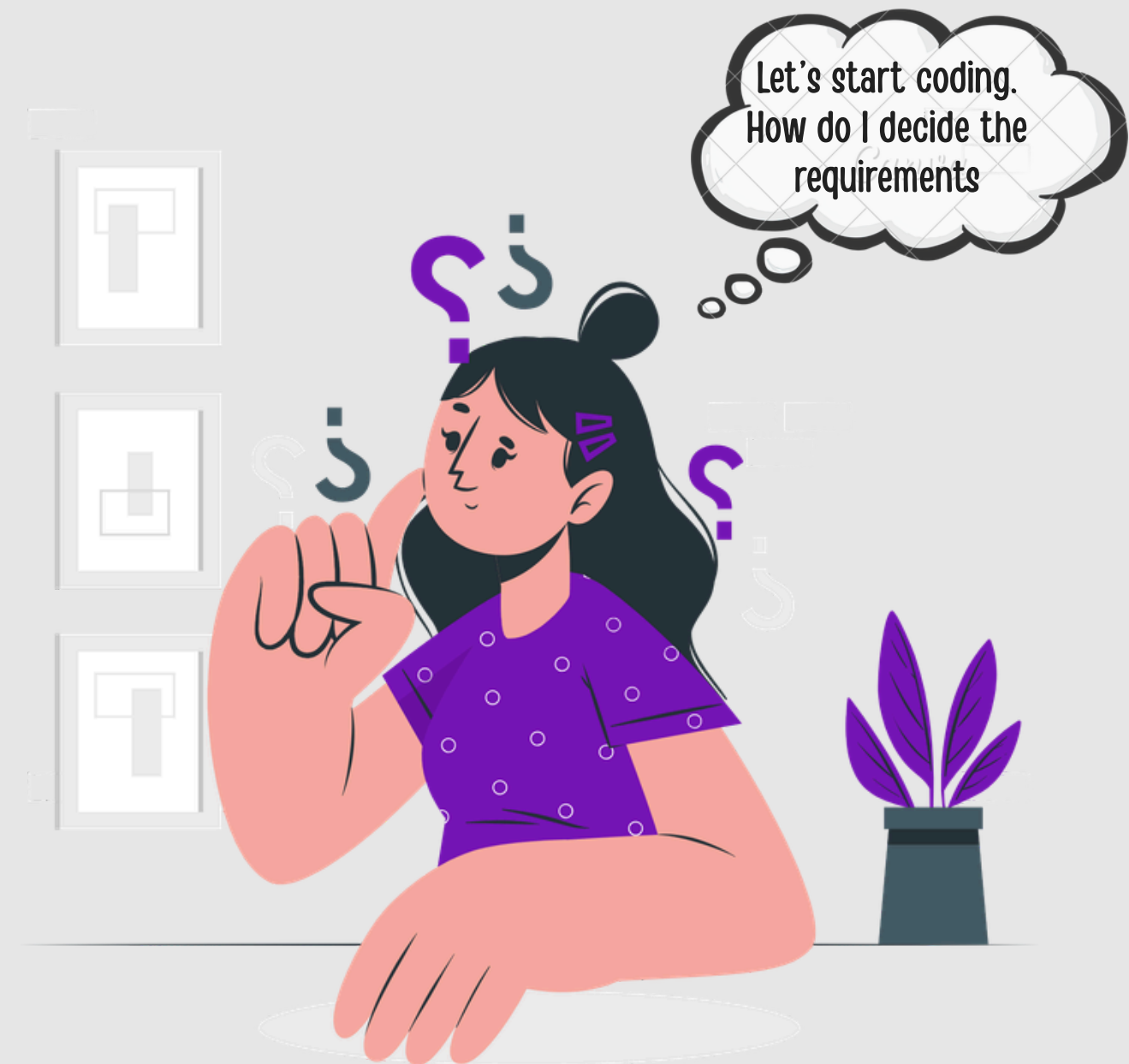
WHAT YOU THINK HAPPENS

Let's start coding immediately!

One person decides everything.

Requirements are clear from beginning

A deployable product in a day



WHAT REALLY HAPPENS

Hours of confusions before
any code

Requirements change every
single time

Multiple teammates with
conflicting ideas

Many milestones, reviews &
sprints

WHAT TO DO?



System architecture

Monolith vs Microservices? REST or GraphQL? How do components communicate? Creates an Architecture Diagram.



Database Design

SQL vs NoSQL? ER diagrams. Schema planning. Relationships and data flow before writing migrations.



Security & Risk

Monolith vs Microservices? REST or GraphQL? How do components communicate? Creates an Architecture Diagram.



Infrastructure / Cloud

AWS, Azure or GCP? Where will it be hosted, scaled & monitored? DevOps pipeline design.



Team Alignment

Git branching strategy (GitFlow). PR review process. Code standards. CI/CD pipeline. Naming conventions.

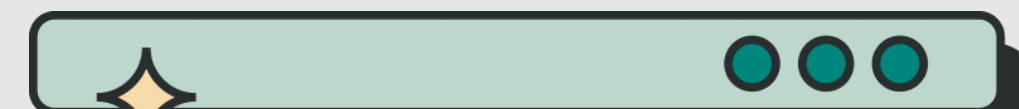


Tech Stack Decision

Why React over Angular? Why PostgreSQL over MongoDB? Documents the reasoning — not just what, but WHY.

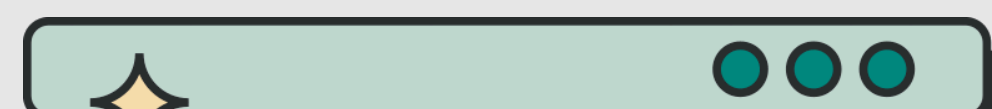
MAKING BETTER **PRODUCTS**

PRODUCT EXCELLENCE IS ACHIEVED THROUGH THREE CORE PILLARS



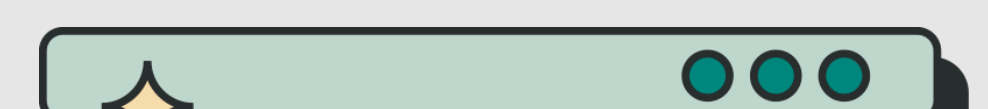
The Right Product

Focusing on strategic alignment and meeting genuine customer needs from the outset.



Done Right

Ensuring technical excellence, high quality, and robust software requirements.



Managed Right

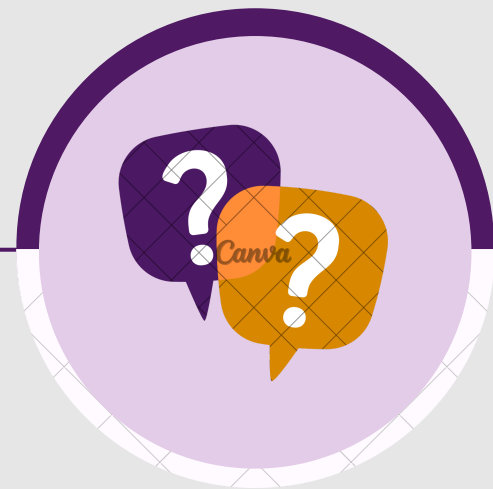
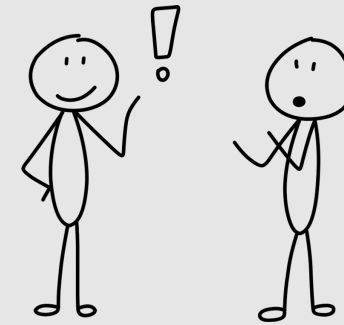
Efficient execution and continuous management of the requirement lifecycle.



UNDERSTANDING REQUIREMENTS

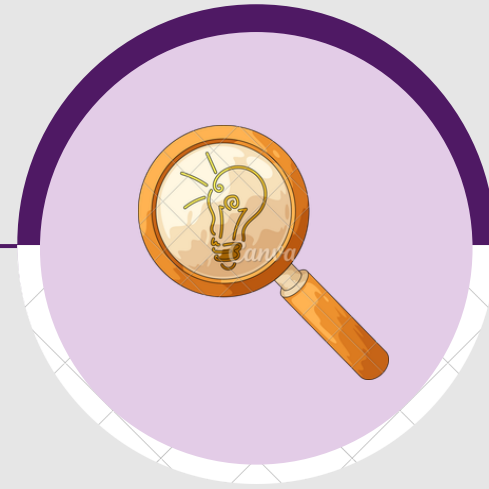


FIGURE OUT WHAT TO BUILD **FIRST**



BUSINESS NEEDS

Identify the market gap.
Think from user/client
POV.



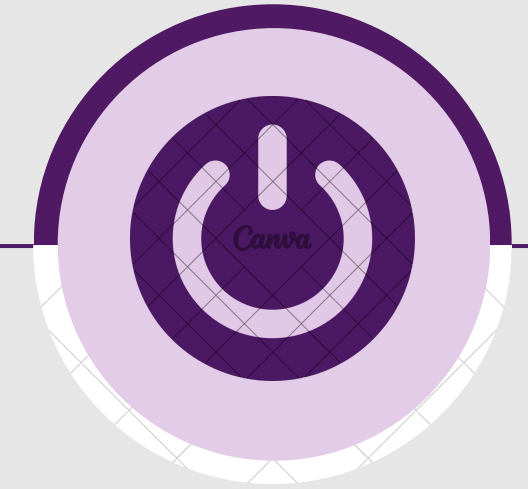
DISCOVERY PHASE

What do you need? Who
uses it? What does
success look like? Output:
Project Brief / SOW.



FEASIBILITY & ESTIMATION

Evaluate: Can we build
this? How long? With
what team? Output:
Timeline, tech stack.



SIGN-OFF

Approved scope, budget &
timeline. Only now does
serious technical work
begin.

QUESTIONS TO **ASK**

GOAL

Understand the **problem**

ASK about **Product**

- What is it?
- Who is it for?
- Why do they need it?
- When is it available?
- Where is it available?
- How does it work?

DELIVERABLE

Document outlining problems of the project.

GOAL

Understand the **people**

ASK : for **Clients**

- Who is the client?
- What are their pain points?
- What do they wish to gain from the project?
- What is motivating them to build this product?

DELIVERABLE

Document outlining who the client is, who the user is, and what their motivations are.

ASK : for **Users**

- Who are the potential users?
- What are their pain points?
- How do they solve the problem currently?
- What are the other products they use?

REQUIREMENTS : THE MOST CRITICAL PHASE

Functional Requirements

- **Describes** what the system should do, i.e., specific functionality or tasks.
- **Focuses** on the behavior and features of the system.
- **Defines** the actions and operations of the system.
- **User authentication** data input/output, transaction processing

VS

Non-Functional Requirements

- **Describes** how the system should perform, i.e., system attributes or quality.
- **Focuses** on the performance, usability, and other quality attributes.
- **Defines** constraints or conditions under which the system must operate
- **Scalability** security, response time, reliability, maintainability.

PRIORITIZING REQUIREMENTS



MSCW PRIORITIZATION

M

MUST HAVE

Non-negotiable product needs that are mandatory for the team

S

SHOULD HAVE

Important initiative that are not vital, but add significant value.

C

COULD HAVE

Nice to have initiatives that will have a small impact if left out.

W

WOULD HAVE

Initiative that are not priority for this time frame.



VALUE VS EFFORT MATRIX



Quick Wins (High Value / Low Effort)

Do these first. Maximum benefit for minimum work.

Big Bets (High Value / High Effort)

Plan carefully. Needs more strategic thinking and planning.

Fill-ins (Low Value / Low Effort)

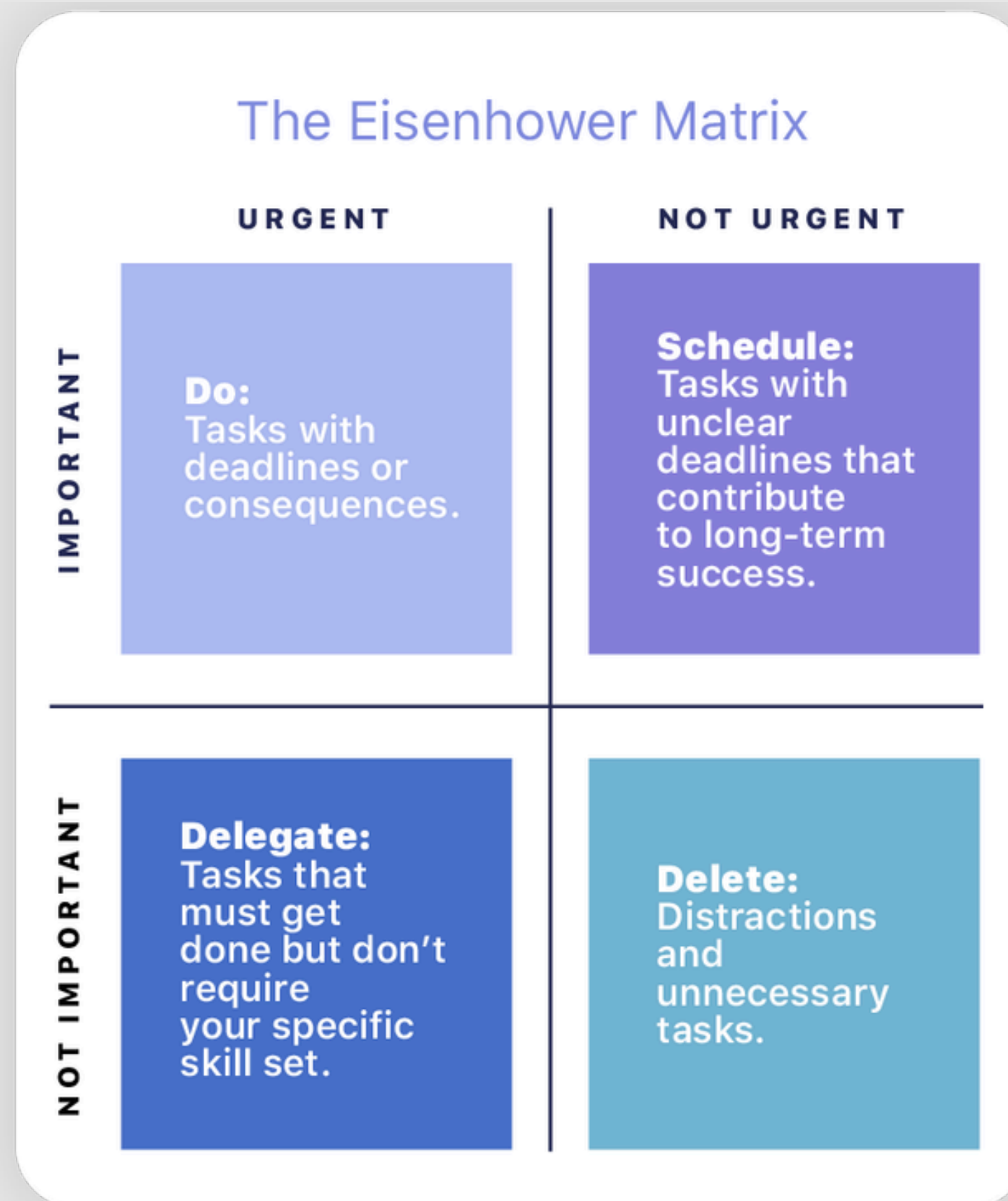
Do later. Tackle these during gaps in major projects.

Money Pit (Low Value / High Effort)

Avoid. Thankless tasks that consume resources.



EISENHOWER MATRIX



HANDS ON ACTIVITY



HANDS ON ACTIVITY

THINK OF A PROJECT **IDEA**

- Define functional and non-functional requirements.
- Use MSCW, value-effort matrix and Eisenhower matrix to classify the functional requirements.

USER STORIES AND USER PERSONAS



BUILDING USER STORIES



EMPATHISE

Learn about the audience
for whom you are
designing.

Who are you designing for?



DEFINE

Pick a point-of view that
focuses on users need

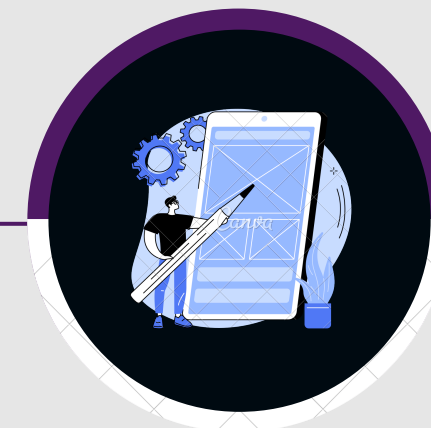
What problem are you solving?



IDEATE

Generate as many
creative solutions as
possible

Quantity over quality



PROTOTYPE

Turn ideas into tangible
forms

Fail faster, learn faster

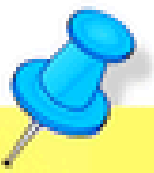


TEST

Evaluate prototypes with
real users.

*Let your users experience the
prototype*

USER STORY



As a <role>
I want <goal>
so that <benefit>

Acceptance criteria:
(Conditions of Satisfaction)

...

...

As an Account Manager
I want a sales report of my account
to be sent to my inbox daily
So that I can monitor the sales
progress of my customer portfolio

Acceptance criteria:

1. The report is sent daily to my inbox
2. The report contains the following sales details: ...
3. The report is in csv format.

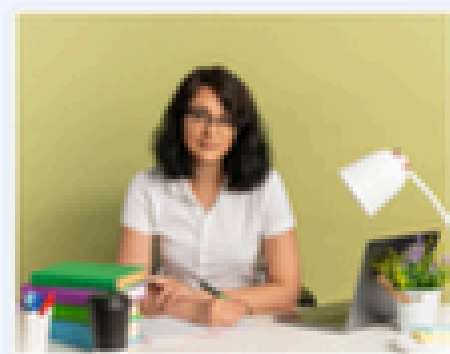
USER PERSONA

A user persona is a semi-fictional representation of your ideal user, based on research.

- Name, photo, age, occupation
- Goals and needs
- Pain points
- Motivations
- Technology usage
- Behavioral traits



USER PERSONA



Riya Shahi

"The hardest part of a hackathon can be finding who to hack with"

Age: 20

Occupation: Computer Engineering Student

Address: Kathmandu

Hackathon experience: 1-2

Bio

Riya Shahi is an aspiring computer engineering student who has recently started her coding journey. Having participated in her very first hackathon recently, Riya found the experience both exciting and challenging. As she was a beginner, she was not quite sure what to expect and was trying to find teammates & properly collaborate with her team. Her team also had trouble with meeting the deadline. Riya has a desire to be confident coder and team player and is actively seeking guidance and strategies to better navigate future hackathons and make meaningful contributions to projects.

growth mindset

tech enthusiast

passionate

creative

Motivations

- skill development and refining
- gaining experience/ new ideas
- growing network

Goals/Expectations

- completing project within given time frame
- learning new skills
- get familiar with working on a team project

Happy moments

- meeting new people
- swags and food
- supportive environment
- fun games and refreshing activities

Pain Points

- time limitation
- finding team members
- originality of code
- collaborating with the team members
- accommodations and refreshments

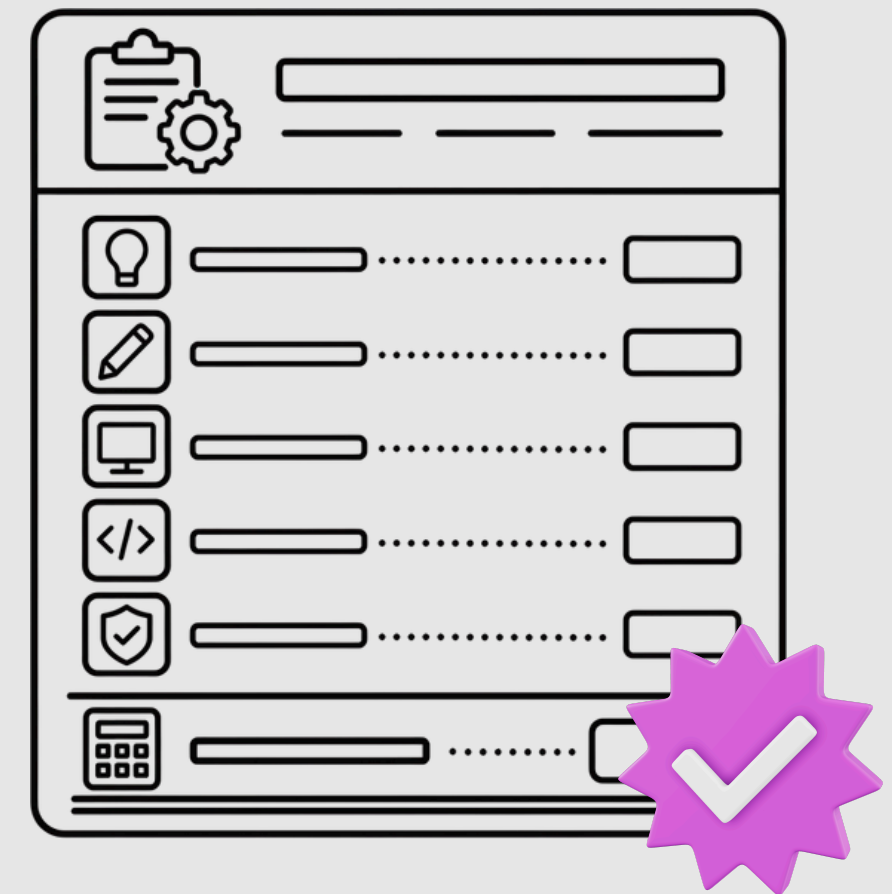
HANDS ON ACTIVITY



THERE IS A PROJECT **UBER**

Create 2 user persona of different age group for an application call uber-a ride sharing app.

ESTIMATION AND PLANNING

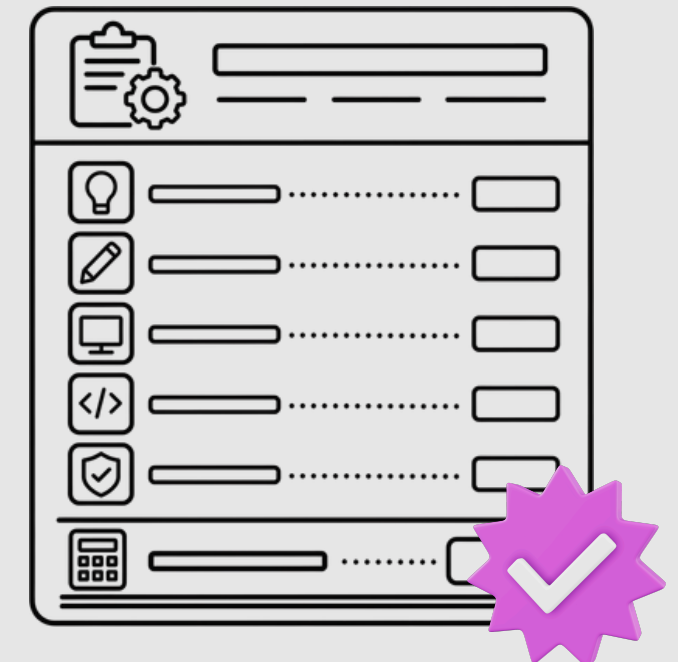


WHAT IS ESTIMATION?

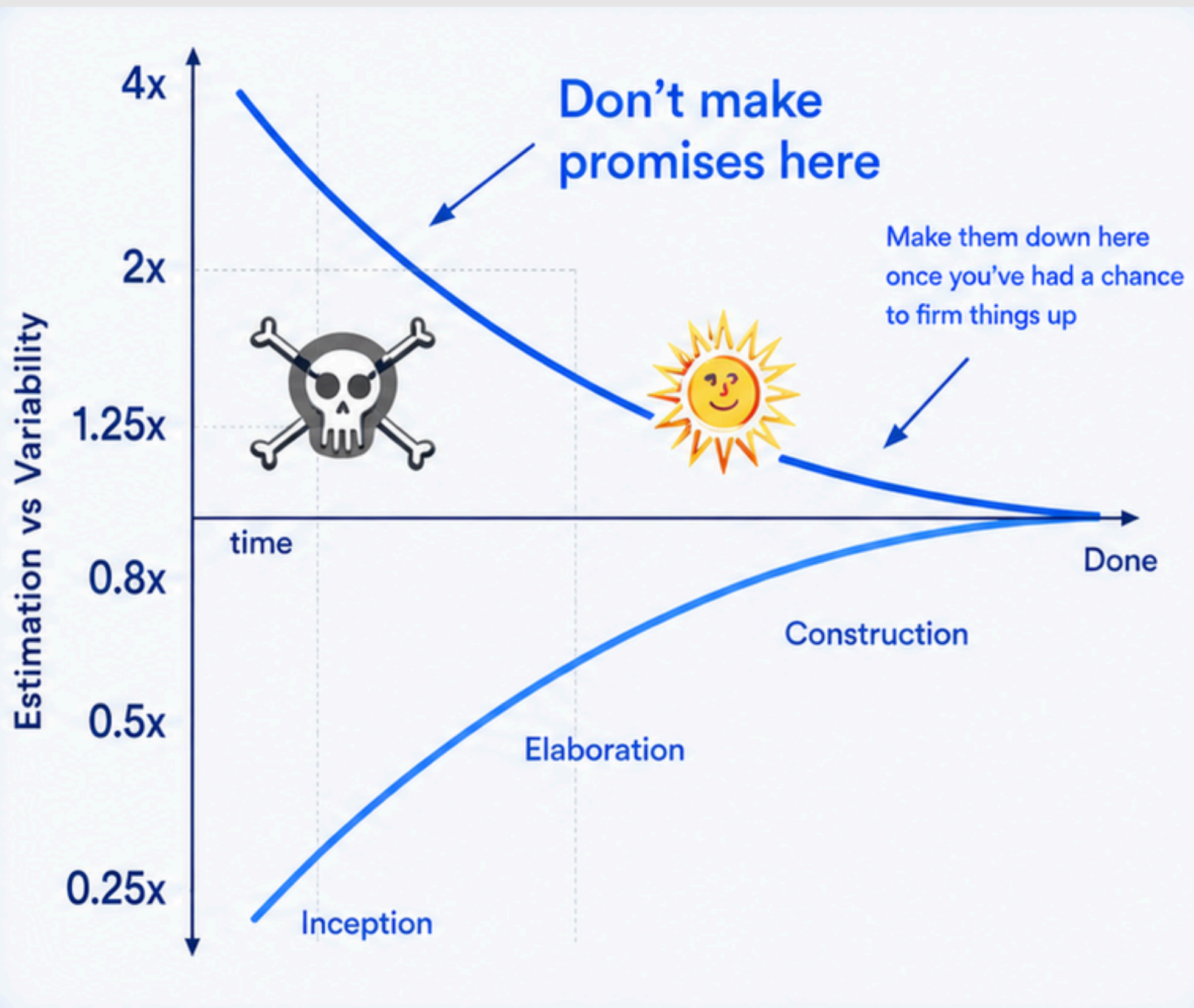
DEFINITION : An estimate is a best guess of how much a job (project) might cost.

INCLUDES : scope, time-frames, budget, and risks.

REQUIRES : breaking it down into individual tasks and figuring out which team members will be doing what.



With estimates being an important key to winning projects, and winning client trust, it is of extreme importance that our estimates are “as accurate as they can be.”



ABSOLUTE VS RELATIVE ESTIMATE

Absolute Estimate

- Unit based. (eg. Hours, days)
- Accounts individual effort.
- Focuses on coordination and control
- Breeds unrealistic accountability.
- Takes longer time and creates debates.

VS

Relative Estimate

- Unit-less. Based on other tasks.
- Promotes collaboration & communication.
- Promotes inspection and adaptation.
- Keeps focus on the team velocity.
- Surprisingly quicker and more accurate.

BASELINE

- A baseline is a reference point or benchmark (a known task/story).
- Team members compare new items to the baseline.
- Assign story points based on relative complexity or effort.
- Enables relative, abstract estimation instead of precise predictions.
- Makes comparison and prioritization easier across tasks.
- Reduces reliance on inaccurate absolute (hour-based) estimates.



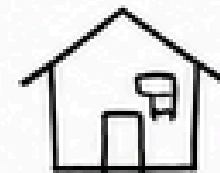
➤ ABSOLUTE vs RELATIVE ESTIMATION ➤



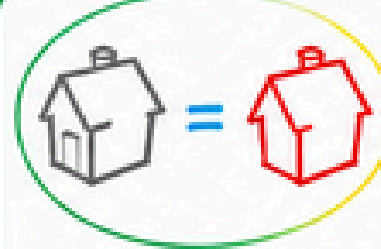
ABSOLUTE ESTIMATION

Estimating a value independently, without comparing to anything else.

ABSOLUTE



ESTIMATE
DURATION:
3 MONTHS



RELATIVE ESTIMATION

Estimating by comparing to something you already understand.

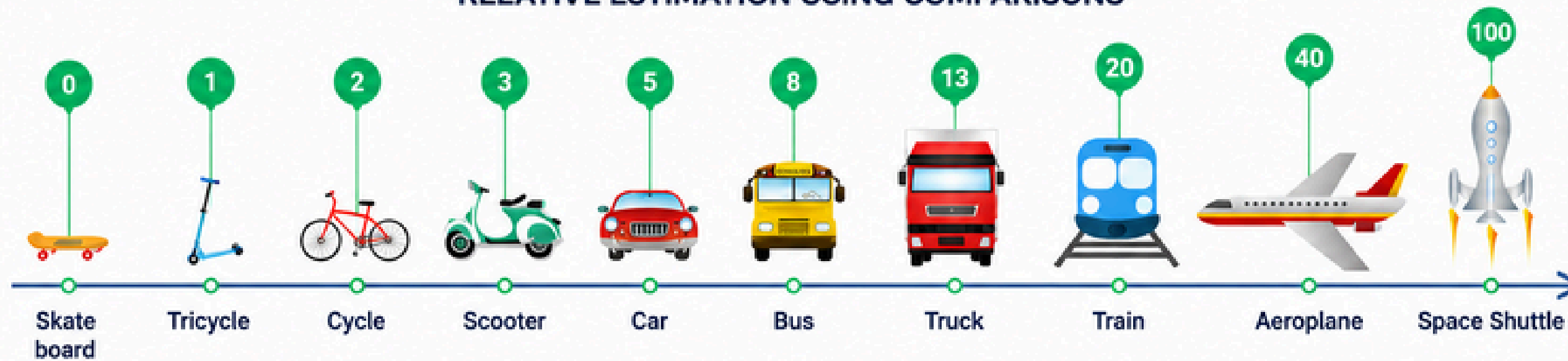


= 2x



RELATIVE

RELATIVE ESTIMATION USING COMPARISONS



SCENARIO: BIRKE IS MOVING TO A NEW APARTMENT

Birke is shifting from his current apartment to a new one across the city. He doesn't have professional movers; he's doing it himself.

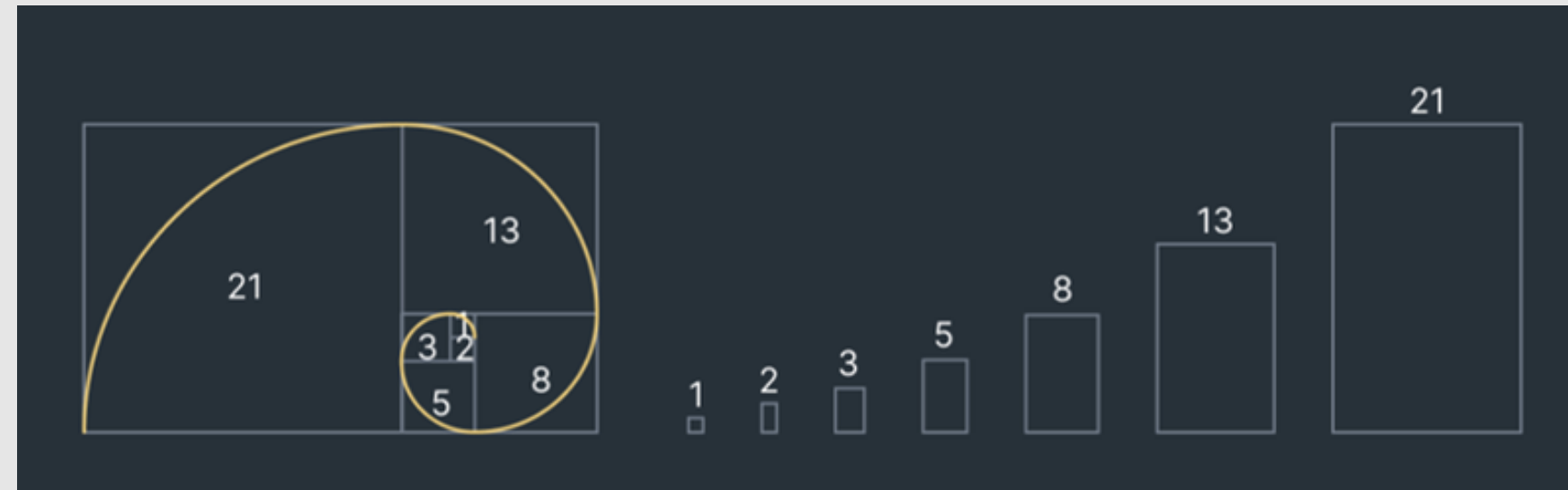
The room will help Harke estimate how long each task might take (in days).

- Sort and declutter belongings
- Buy packing supplies (boxes, tape, bubble wrap)
- Pack kitchen items
- Pack clothes and personal items
- Pack books and electronics
- Label all boxes
- Disassemble furniture (bed, table, shelf, etc.)
- Clean the old apartment
- Load items into the transport vehicle
- Drive to the new apartment
- Unload items at the new apartment
- Reassemble furniture
- Unpack essentials (kitchen, clothes)
- Dispose of packing material
- Organize items in the new apartment
- Have a housewarming party



STORY POINTS & FIBONACCI SEQUENCE

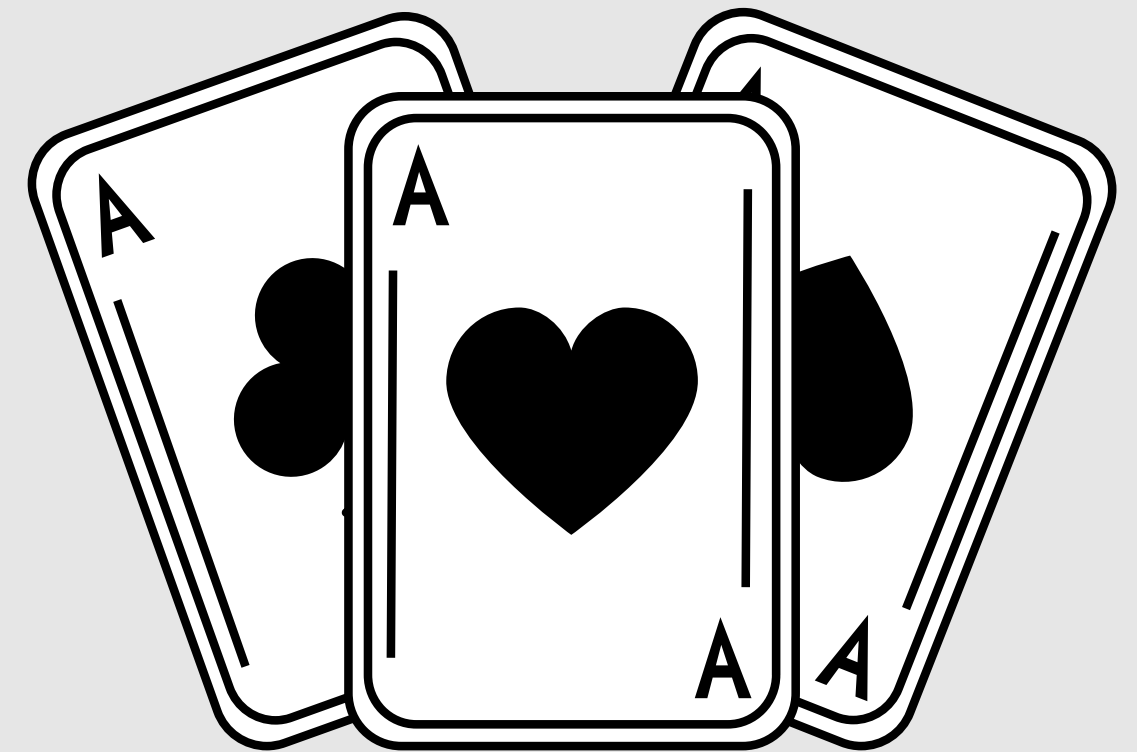
- The Fibonacci Sequence is the series of numbers:
0, 1, 2, 3, 5, 8, 13,...



- The next number is found by adding up the two numbers before it.
- Story point scale is a sequence of numbers that represent the relative effort involved in completing a task or user story, such as 0, 1, 2, 3, 5, 8, 13.

PLANNING POKER

- A consensus-based technique for estimation, mostly used to estimate effort or relative size of development goals in software product development.
- It is done with story points, ideal days, or any other estimating units.
- It helps to estimate the effort for User Stories (Product Backlog Items, Value Drivers)



ACTIVITY:

THE MOMO PARTY

- As a project team, we need to estimate the "Story Points" (effort, complexity, and risk) of the tasks required to host a 15-person MoMo party. We will estimate them treating them like a real Project Backlog.
- The Estimation Scale (Fibonacci): We use relative sizing: 1, 2, 3, 5, 8, 13, 20 (Higher number = more effort, more complexity, or higher uncertainty).
- The Rules of the Game:
 1. We pick each task and , everyone enters their estimated Story Point for that task.
 2. If the numbers don't match, the highest and lowest estimators must explain their reasoning.
 3. Our Baseline Task (Reference Point): Boiling a kettle of water = 1 Story Point

ACTIVITY: **THE MOMO** **PARTY**

SCAN ME



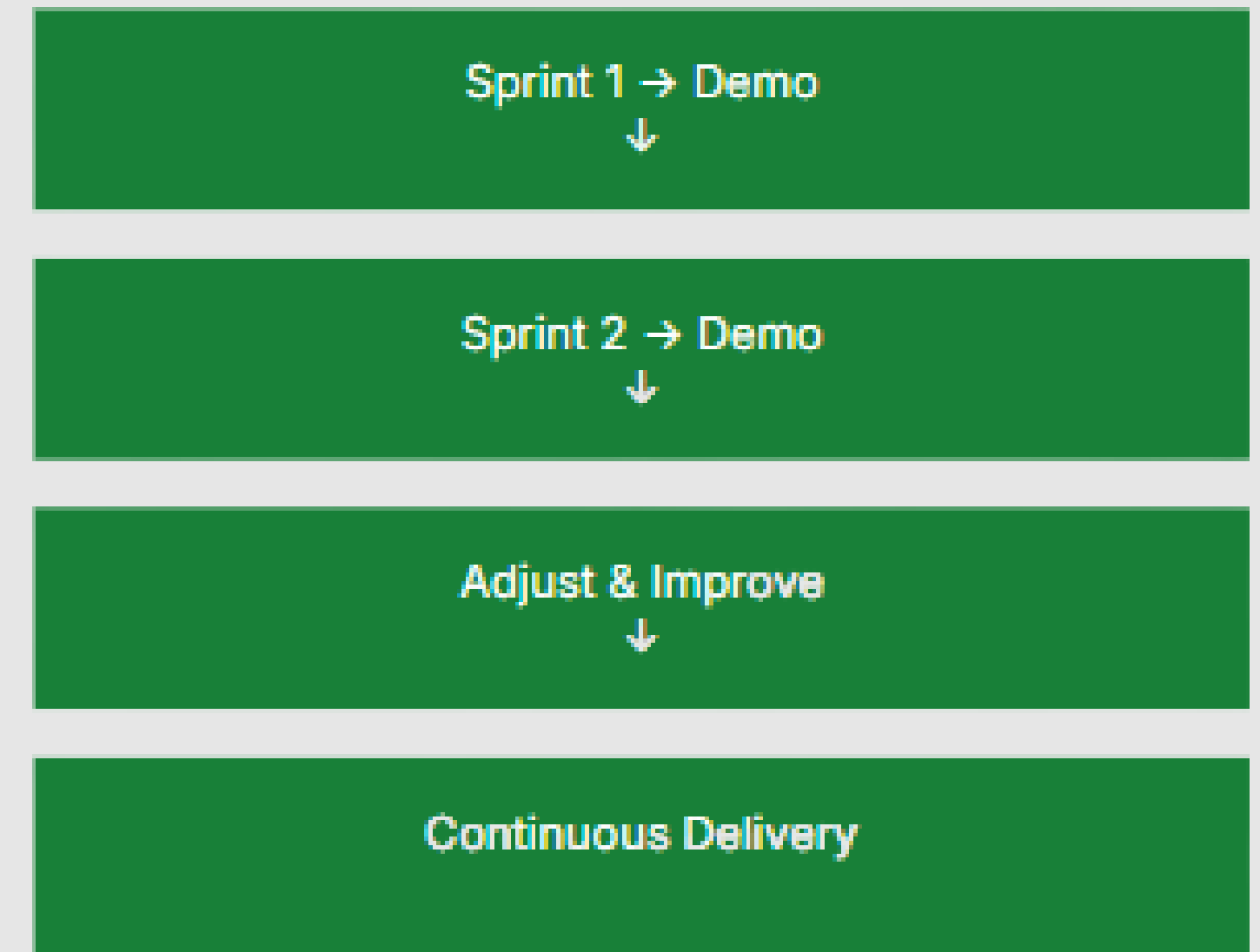
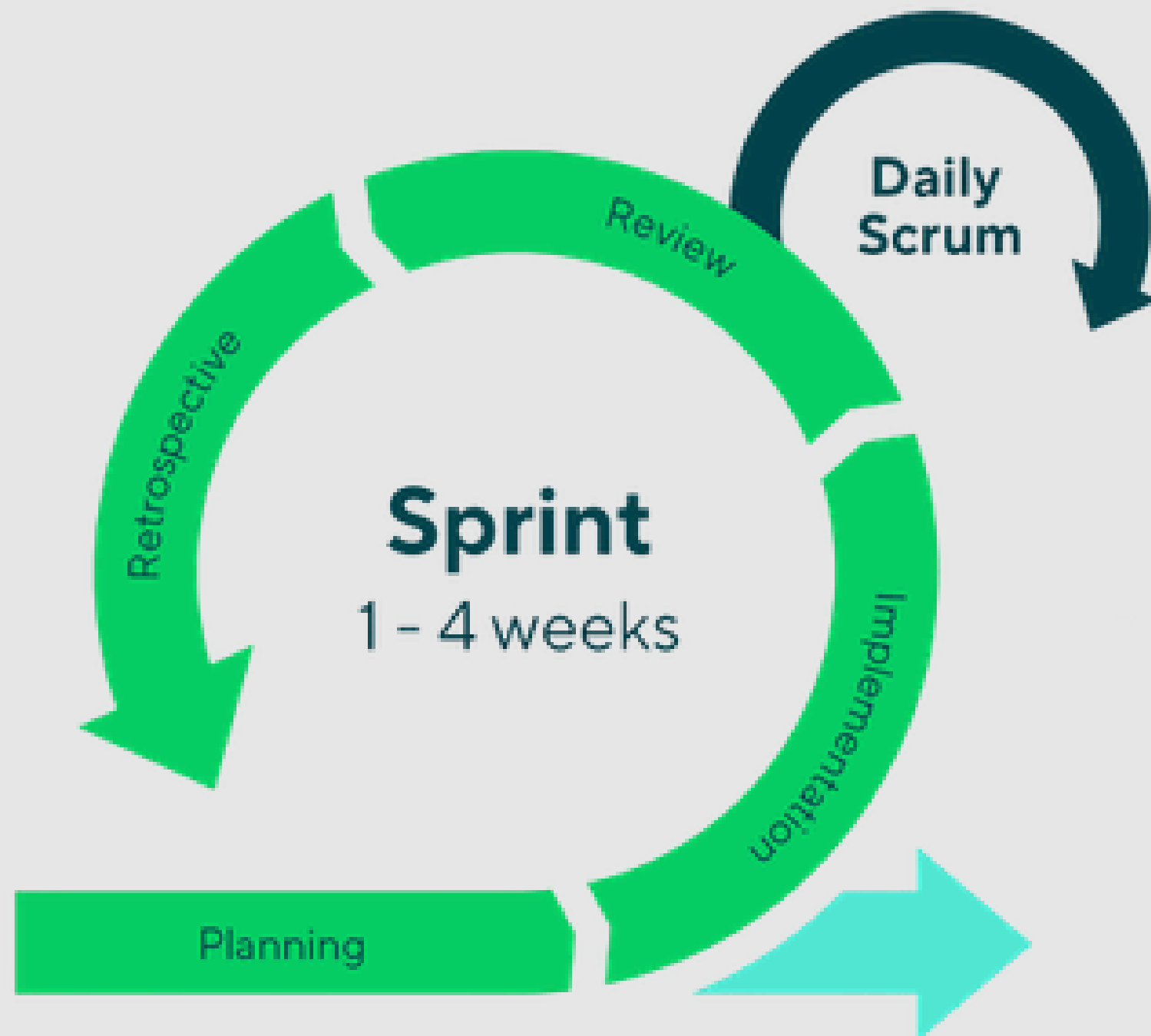


Leapfrog
Student Partnership
Program

AGILE & TOOLS USED



AGILE: HOW MODERN TEAMS ACTUALLY WORK



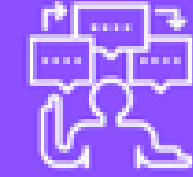
PLANNING AND EXECUTION OF SPRINT



Define the sprint goal.



Create the sprint backlog.



Hold a sprint planning meeting.



Start the sprint.



Hold daily stand-up meetings.



Monitor progress.



Hold a sprint review meeting.



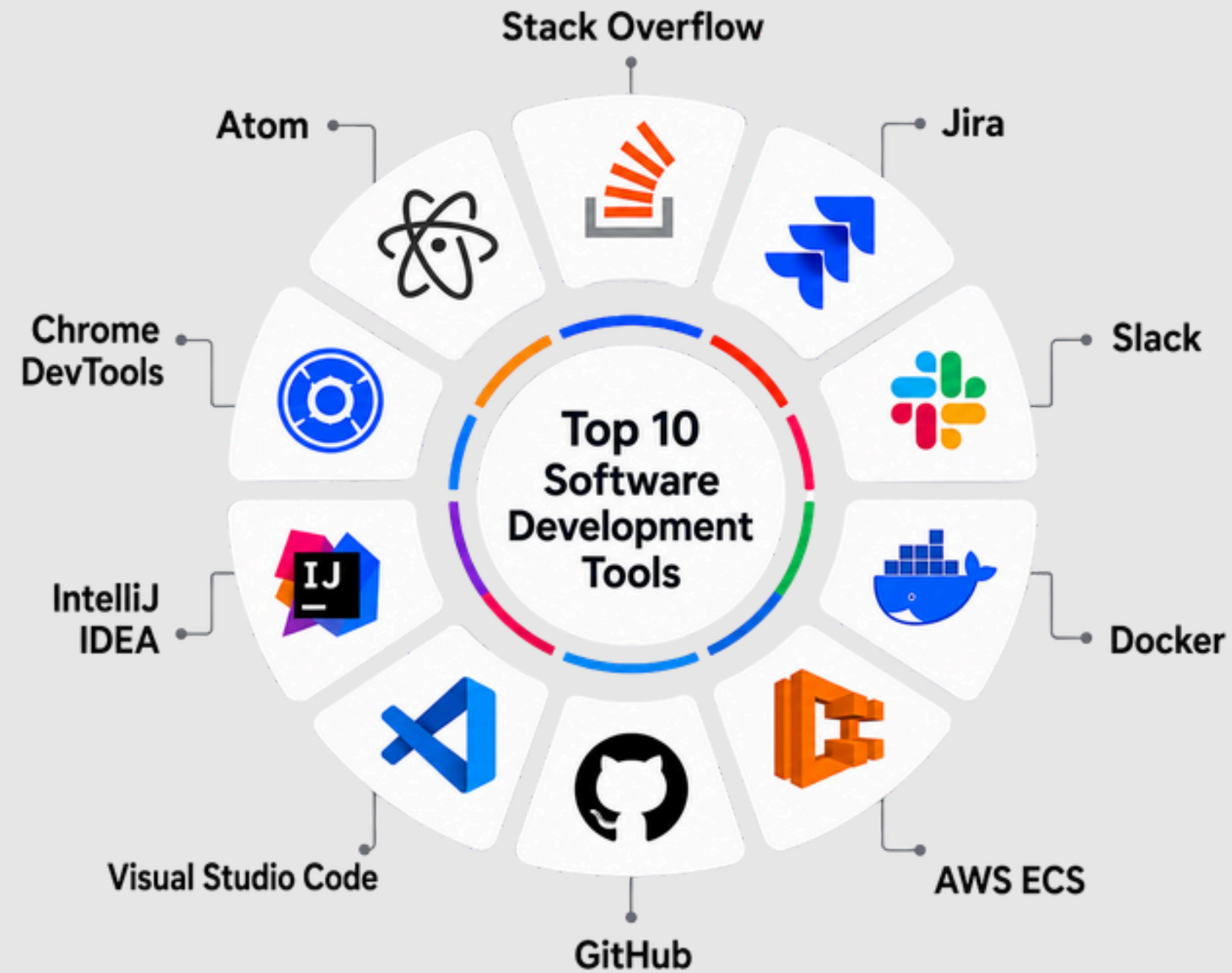
Hold a sprint retrospective meeting.



Repeat the process.



TOOLS USED IN THE INDUSTRY

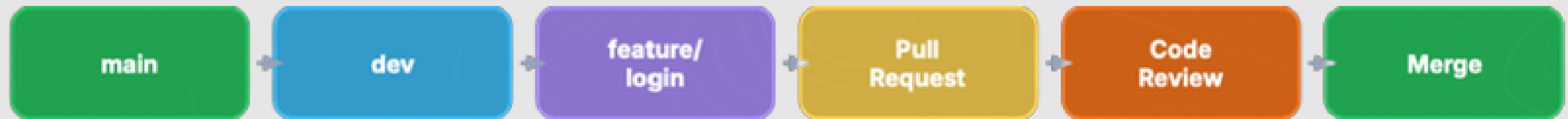




GITHUB WORKFLOW



GITHUB WORKFLOW



1 Create a Branch

```
git checkout -b feature/login
```

Always branch off dev. Never commit directly to main.

2 Commit Your Work

```
git commit -m "feat: add login form"
```

Commit often. Small, focused commits are easier to review.

3 Push & Open a PR

```
git push origin feature/login
```

Open a Pull Request on GitHub. Describe what changed and why.

4 Code Review

```
# reviewer leaves comments
```

A teammate checks logic, security & style. Respond to feedback professionally.

5 Merge to dev

```
Merge PR → dev branch
```

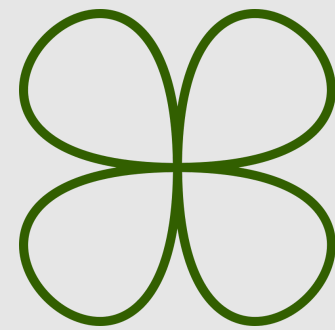
After approval, merge. Delete the feature branch. Dev gets deployed to staging.

6 Release to main

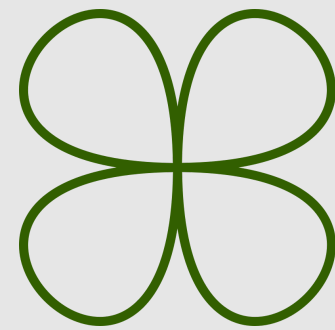
```
dev → main (release PR)
```

After QA on staging, merge dev into main. This triggers production deploy.

ANY QUESTIONS?



THANK YOU



FEEDBACK FORM



THE FORM CAN BE FILLED ANONYMOUSLY.
PLEASE PROVIDE HONEST RATINGS.
THE FORM IS ONLY ACCESSIBLE BY THE LSPP TEAM.